

GUIDA PER IL FORMATORE

Capitolo 7

TROUBLESHOOTING

Identificazione e Risoluzione di Problemi Comuni

Corso di Formazione Professionale
Claude Desktop e Claude for Code

Livello: Intermedio | Bilanciamento: 30% Teoria / 70% Pratica

Indice

Indice	2
1. FONDAMENTI TEORICI	5
1.1 Principi del Troubleshooting per AI Assistants	5
1.2 Categorie di Problemi Comuni	5
Categoria 1: Problemi di Comprensione	5
Categoria 2: Problemi di Qualità dell'Output	6
Categoria 3: Problemi di Inconsistenza	6
Categoria 4: Problemi di Appropriatezza del Task	6
Categoria 5: Problemi di Performance e Degradazione	6
1.3 Framework di Root Cause Analysis	7
Step 1: Caratterizzazione del Sintomo	7
Step 2: Raccolta del Contesto	7
Step 3: Formulazione di Ipotesi	7
Step 4: Testing Sistemático	7
Step 5: Applicazione della Soluzione	7
Step 6: Prevenzione	7
1.4 Relazione con Spec-Driven Development	8
2. DIMOSTRAZIONI PRATICHE	8
2.1 Scenario 1: Claude Misunderstanding	8
Prompt Problematico	8
Prompt Migliorato	8
2.2 Scenario 2: Low-Quality Output	8
Lezione Chiave	8
2.3 Scenario 3: Context Window Issues	9
Tre Soluzioni	9
2.4 Scenario 4: Task Complexity - Scomposizione Efficace	9
Setup dello Scenario	9
Sintomi Osservabili	9
Processo Diagnostico	9
Soluzione Applicata	9
Varianti e Pattern Correlati	10
2.5 Scenario 5: Iterative Refinement - Pattern Ottimali vs Inefficaci	10
Setup dello Scenario	10
Best Practice: Iterazione Efficace	10
Principi di Iterazione Efficace	11

Template per Feedback Efficace.....	11
3. ESERCIZI HANDS-ON.....	12
3.1 Esercizio 1: Diagnosi di Prompt Ambigui	12
3.2 Esercizio 2: Context Window Management.....	12
Obiettivi di Apprendimento	12
Tempo Stimato	12
Scenario dell'Esercizio	12
Task.....	12
Criteri di Successo	13
Note per il Formatore - Discussione Post-Esercizio	13
3.3 Esercizio 3: Debugging Spec-Driven Workflows	13
Obiettivi di Apprendimento	13
Tempo Stimato	13
Scenario dell'Esercizio	13
Task per Ciascun Caso.....	14
Caso A - Specifica Problematica.....	14
Caso B - Implementazione Divergente.....	14
Criteri di Successo	14
3.5 Esercizio 5: Real-World Challenge (Livello Avanzato)	14
Obiettivi.....	14
Tempo	14
Challenge.....	15
Task Suggestiti (se non avete preferenze).....	15
Criteri di Valutazione	15
4. CASI D'USO REALI	16
4.1 Case Study: Document Generation con Formatting Inconsistente	16
Background e Contesto.....	16
Journey Diagnostico	16
Soluzione Implementata.....	16
Risultati.....	17
Lezioni Apprese	17
4.2 Case Study: Code Generation con Repeated Errors	17
Background e Contesto.....	17
Journey Diagnostico	17
Soluzione	17
Risultati.....	17

4.4 Case Study: Multi-Step Workflow Failures	18
Background.....	18
Diagnosi.....	18
Soluzione	18
5. BEST PRACTICES E LINEE GUIDA.....	19
5.1 Framework Diagnostico Completo	19
5.2 Pratiche Preventive	19
5.3 Decision Trees per Troubleshooting Rapido	20
7. NOTE PER IL FORMATORE	21
7.4 Domande Comuni e Risposte Prepare.....	21
7.5 Follow-Up e Homework	21
8. RISORSE SUPPLEMENTARI.....	22
8.1 Quick Reference Card	22
8.5 Glossario Troubleshooting.....	22

1. FONDAMENTI TEORICI

Il troubleshooting nell'utilizzo di assistenti AI come Claude rappresenta una competenza fondamentale per ogni professionista che intende sfruttare appieno le potenzialità di questi strumenti. Questa sezione fornisce le basi teoriche necessarie per comprendere la natura dei problemi che possono emergere e sviluppare un approccio sistematico alla loro risoluzione.

1.1 Principi del Troubleshooting per AI Assistants

Il troubleshooting quando si lavora con Claude richiede un cambio di mentalità rispetto al debugging tradizionale di software. Non si tratta di trovare bug nel codice, ma di comprendere e risolvere problemi di comunicazione, contesto e aspettative tra utente umano e sistema AI.



Principio Fondamentale

Il troubleshooting con Claude è essenzialmente un problema di comunicazione: il 90% dei problemi deriva da ambiguità nelle richieste, contesto insufficiente, o aspettative non allineate alle capacità effettive dello strumento.

I principi cardine del troubleshooting efficace includono:

- **Approccio Sistematico:** Seguire una metodologia strutturata anziché procedere per tentativi casuali. Questo include identificare i sintomi, formulare ipotesi sulle cause, testare sistematicamente le ipotesi, e applicare soluzioni mirate.
- **Isolamento del Problema:** Identificare con precisione dove si manifesta il problema per evitare di applicare soluzioni generiche che non affrontano la root cause.
- **Iterazione Controllata:** Modificare un elemento alla volta e osservare l'impatto, piuttosto che cambiare multipli fattori simultaneamente.
- **Documentazione delle Soluzioni:** Registrare sistematicamente quali problemi si incontrano e quali soluzioni funzionano.
- **Prevenzione Proattiva:** Applicare best practices fin dall'inizio per evitare problemi comuni prima che si manifestino.

1.2 Categorie di Problemi Comuni

I problemi nell'utilizzo di Claude si raggruppano in categorie distinte, ciascuna con caratteristiche specifiche e strategie di risoluzione appropriate.

Categoria 1: Problemi di Comprensione

Questa categoria include tutte le situazioni in cui Claude interpreta la richiesta in modo diverso da quanto l'utente intendeva.

- Risposte che risolvono un problema diverso da quello posto
- Output in formato o struttura non richiesti
- Assunzioni errate su contesto o requisiti

Root Causes Comuni:

- Ambiguità nel linguaggio utilizzato
- Contesto insufficiente

- Assunzioni implicite non esplicitate

Categoria 2: Problemi di Qualità dell'Output

Questa categoria comprende situazioni in cui Claude comprende la richiesta ma produce output di qualità inferiore alle aspettative.

- Contenuto superficiale o generico
- Errori fattuali o imprecisioni tecniche
- Stile inappropriato
- Lunghezza inadeguata

Categoria 3: Problemi di Inconsistenza

Risposte successive che mostrano incoerenze o contraddizioni.

- Risposte che contraddicono informazioni date precedentemente
- Variazioni di qualità tra iterazioni
- Claude 'dimentica' istruzioni fornite

Categoria 4: Problemi di Appropriatezza del Task

Situazioni in cui il task richiesto non è appropriato per Claude.

- Necessità di numerose iterazioni anche per task semplici
- Output che variano significativamente ad ogni tentativo
- Claude 'lotta' visibilmente con il task

Categoria 5: Problemi di Performance e Degradazione

Problemi che emergono nel tempo, specialmente in conversazioni lunghe.

- Qualità che progressivamente peggiora
- Risposte che diventano più lente
- Claude perde traccia di elementi discussi prima

1.3 Framework di Root Cause Analysis

Un approccio sistematico alla root cause analysis è fondamentale per troubleshooting efficace.

Step 1: Caratterizzazione del Sintomo

Descrivere con precisione cosa non sta funzionando:

- Qual è esattamente il comportamento osservato?
- Qual era il comportamento atteso?
- Il problema è consistente o intermittente?

Step 2: Raccolta del Contesto

- Esaminare il prompt esatto utilizzato
- Verificare cosa è incluso nel contesto
- Valutare la lunghezza della conversazione
- Identificare eventuali modifiche recenti

Step 3: Formulazione di Ipotesi

Basandosi sui sintomi, formulare ipotesi sulla possibile causa usando le 5 categorie.

Step 4: Testing Sistematico

- Identificare un test specifico per ciascuna ipotesi
- Eseguire il test in condizioni controllate
- Osservare e confrontare risultati

Step 5: Applicazione della Soluzione

- Implementare la correzione mirata
- Verificare che risolva il problema
- Controllare effetti collaterali
- Documentare la soluzione

Step 6: Prevenzione

- Identificare pattern ricorrenti
- Aggiornare template personali
- Condividere con il team

1.4 Relazione con Spec-Driven Development

Lo spec-driven development previene proattivamente molti problemi comuni attraverso la definizione rigorosa di specifiche prima dell'implementazione.

Insight Chiave

Lo spec-driven development trasforma il troubleshooting da arte oscura a processo sistematico. Quando emerge un problema, la specifica fornisce un 'contratto' chiaro rispetto al quale valutare l'implementazione.

2. DIMOSTRAZIONI PRATICHE

Questa sezione fornisce script completi per dimostrazioni live di troubleshooting. Ogni scenario include setup dettagliato, sintomi osservabili, processo diagnostico step-by-step, e risoluzione.

Nota per il Formatore

Queste dimostrazioni dovrebbero essere eseguite live, non semplicemente descritte. L'obiettivo è mostrare il processo di troubleshooting in azione.

2.1 Scenario 1: Claude Misunderstanding

Dimostra come prompt ambigui causano misunderstanding e come risolverli con specificità.

Prompt Problematico

"Crea un report sulle vendite con i dati che ti ho dato."

Prompt Migliorato

"Analizza i dati di vendita e crea un report trimestrale con: 1) Executive Summary, 2) Analisi per Regione, 3) Analisi per Prodotto, 4) Trend Mensili, 5) Raccomandazioni. Tone professionale per management senior."

2.2 Scenario 2: Low-Quality Output

Dimostra come specifiche di qualità esplicite migliorano drasticamente l'output.

Lezione Chiave

Specificare requisiti tecnici E requisiti di qualità, non solo funzionalità. L'approccio spec-driven è particolarmente efficace per codice production-ready.

2.3 Scenario 3: Context Window Issues

Dimostra come riconoscere e risolvere problemi di context saturation.

Tre Soluzioni

1. Reset con Summary: Chiedere a Claude un summary strutturato e iniziare nuova conversazione
2. Utilizzo di Progetti: Creare progetto con context document persistente
3. Decomposizione: Scomporre task complesso in componenti indipendenti

2.4 Scenario 4: Task Complexity - Scomposizione Efficace

Setup dello Scenario

Contesto: Un product manager vuole che Claude crei una roadmap prodotto completa per i prossimi 12 mesi, includendo analisi competitiva, prioritizzazione features, timeline, resource allocation, e risk assessment.

Sintomi Osservabili

- Dopo 5-6 iterazioni, il risultato è ancora frammentario e incompleto
- Ogni iterazione produce miglioramenti marginali ma non convergenza verso soluzione soddisfacente
- Claude produce analisi superficiali per ciascun componente
- Il tentativo di gestire tutto in una conversazione crea confusione crescente

Processo Diagnostico

Step 1: Riconoscere il Pattern

"Questo è un classico caso di task inappropriately complex per approccio monolitico. Non è che Claude non possa fare roadmap - è che stiamo chiedendo troppo in un'unica richiesta. Una roadmap completa richiede multipli tipi di analisi, ciascuno con depth significativo."

Step 2: Identificare Componenti Naturali

La roadmap può essere decomposta in componenti logicamente indipendenti.

Soluzione Applicata

Approccio Multi-Conversazione Strutturato

Con questo approccio:

- Ogni componente riceve attenzione focalizzata e depth appropriato
- Claude può applicare reasoning sofisticato a ciascun sotto-problema
- I risultati di ciascuna fase informano le fasi successive
- Il deliverable finale è molto più robusto e actionable
- Tempo totale effettivamente minore rispetto a iterazioni infruttuose su approccio monolitico

✓ Best Practice - Riconoscimento di Task Complessi

Regola empirica: Se il deliverable richiesto ha 4+ componenti distinti che ciascuno potrebbe essere un documento standalone, considerare decomposizione. Segnali: prompt iniziale supera 150 parole, si descrivono multipli 'e anche...' nella richiesta.

Varianti e Pattern Correlati

Questo pattern di decomposizione si applica a molti scenari:

- Business plans completi → Decomposizione in market analysis, financial projections, go-to-market, etc.
- Technical architectures → Decomposizione in data layer, application layer, infrastructure, security, etc.
- Content strategies → Decomposizione in audience analysis, channel strategy, content calendar, metrics framework

Il principio generale: Decomposizione aumenta qualità quando i componenti beneficiano da deep focus separato.

2.5 Scenario 5: Iterative Refinement - Pattern Ottimali vs Inefficaci**Setup dello Scenario**

Contesto: Un content writer ha chiesto a Claude di scrivere un articolo blog. Il primo draft è buono ma necessita refinement. Questo scenario contrasta approcci efficaci vs. inefficaci all'iterazione.

Best Practice: Iterazione Efficace

Approccio sistematico e produttivo:

Iterazione 1 - Feedback Specifico e Strutturato:

"Il draft è un buon starting point. Ecco il feedback specifico per refinement: STRUTTURA: ✓ Intro è coinvolgente X Sezione 2 troppo tecnica per target audience (marketing managers non-technical) X Conclusione non include clear call-to-action CONTENUTO: ✓ Esempi in sezione 1 sono rilevanti X Manca esempio pratico in sezione 3 ✓ Statistiche citate sono convincenti TONE: X Passaggi troppo formali, target audience preferisce conversational tone AZIONI RICHIESTE: 1. Semplifica sezione 2 eliminando jargon tecnico 2. Aggiungi step-by-step example in sezione 3 3. Rewrite conclusion con CTA per download risorsa gratuita 4. Rendi tone più conversational mantenendo credibilità"

Iterazione 2 - Validation Checkpoint:

"Ottimo, le modifiche sono sulla direzione giusta. Un ultimo refinement: - L'esempio in sezione 3 funziona bene ma può essere condensato (attualmente 8 step, ideale 5 step core) - CTA è chiara ma può essere più compelling: invece di 'Scarica la guida' provare angle benefit-focused Fai questi due aggiustamenti finali."

Risultato: 2 iterazioni, feedback chiaro e actionable, convergenza rapida verso output di qualità.

Principi di Iterazione Efficace

Elementi che rendono l'iterazione produttiva:

4. Feedback Specifico: Identificare esattamente cosa funziona e cosa no, non giudizi generici
5. Formato Strutturato: Organizzare feedback in categorie (struttura, contenuto, tone, etc.)
6. Prioritization: Se ci sono multipli issues, indicare quali sono più critici
7. Azioni Chiare: Tradurre feedback in richieste di azione concrete
8. Validation Progressiva: Riconoscere miglioramenti tra iterazioni, non resettare mentalmente
9. Stopping Criteria: Sapere quando 'good enough' evita over-iteration

Segnale di Warning - Iteration Loop

Se dopo 3 iterazioni non si vedono miglioramenti incrementali significativi, il problema è probabilmente nel feedback fornito (troppo vago o contraddittorio) o nel task stesso (aspettative non realistiche). Fermarsi e riconsiderare l'approccio.

Template per Feedback Efficace

Struttura raccomandata per feedback iterativo:

[COSA FUNZIONA BENE] - Elemento positivo 1 - Elemento positivo 2 [COSA NECESSITA MIGLIORAMENTO] - Issue specifico 1 (con esempio) - Issue specifico 2 (con esempio)
[AZIONI RICHIESTE] 1. Azione specifica e actionable 1 2. Azione specifica e actionable 2 3. ...

Questo template guida verso feedback costruttivo e actionable che Claude può interpretare chiaramente.

3. ESERCIZI HANDS-ON

Questa sezione fornisce esercizi pratici progressivi per sviluppare competenze di troubleshooting.

3.1 Esercizio 1: Diagnosi di Prompt Ambigui

Obiettivi: Identificare ambiguità, trasformare prompt vaghi in specifici, comprendere relazione specificità-qualità.

Tempo: 20 minuti (10 min esercizio + 10 min debriefing)

Prompt da Analizzare:

10. "Aiutami con questo documento" [con doc allegato]
11. "Scrivi codice per gestire dati"
12. "Crea una presentazione sul nostro prodotto"

Per ciascuno: identificare ambiguità, riscrivere con specificità, testare entrambe versioni.

3.2 Esercizio 2: Context Window Management

Obiettivi di Apprendimento

- Riconoscere sintomi di context saturation
- Applicare tecniche di context compression
- Decidere quando reset vs. continuare conversazione

Tempo Stimato

30 minuti (20 min esercizio + 10 min debriefing)

Scenario dell'Esercizio

Vi viene fornita una conversazione simulata lunga (transcript di 25+ messaggi) dove un utente sta lavorando su un progetto software complesso con Claude. La conversazione mostra evidenti segni di degradazione.

Task

Parte A - Diagnosi (10 minuti):

13. Leggere il transcript e identificare almeno 3 sintomi specifici di context saturation
14. Indicare approssimativamente a quale punto della conversazione iniziano a manifestarsi problemi
15. Identificare quale informazione è critica e deve essere preservata vs. quale è 'rumore'

Parte B - Soluzione (10 minuti):

16. Creare un summary strutturato delle informazioni essenziali (max 200 parole)
17. Formulare il primo prompt di una nuova conversazione che include il summary e porta avanti il lavoro

18. Identificare quali file/materiali dovrebbero essere caricati nella nuova conversazione

Criteri di Successo

- Summary cattura tutte le decisioni architettureali chiave
- Summary è autocontenuto (comprensibile senza leggere tutta la conversazione precedente)
- Nuovo prompt chiaramente indica next step da compiere
- Partecipante sa spiegare perché hanno escluso certe informazioni dal summary



Hint

Nel creare il summary, concentratevi su: decisioni prese (non le discussioni che hanno portato alle decisioni), stato corrente del deliverable, requisiti/vincoli chiari, next actions. Evitate: conversazioni esplorative risolte, alternative considerate e scartate, debugging di problemi già risolti.

Note per il Formatore - Discussione Post-Esercizio

Durante il debriefing, confrontare diversi approcci dei partecipanti:

- Quali informazioni diversi partecipanti hanno considerato 'essenziali' vs. 'rumore'? Perché?
- Quanto sono lunghi i summary creati? (Se superano 300 parole, probabilmente includono troppo dettaglio)
- I nuovi prompt mantengono momentum o 'ricominciano da zero'?

Enfatizzare: Context management è skill che si sviluppa con pratica. All'inizio si tende a voler preservare troppo.

3.3 Esercizio 3: Debugging Spec-Driven Workflows

Obiettivi di Apprendimento

- Distinguere tra problemi nella specifica vs. problemi nell'implementazione
- Fornire feedback mirato su implementazioni spec-driven
- Iterare efficacemente su specifiche problematiche

Tempo Stimato

35 minuti (25 min esercizio + 10 min debriefing)

Scenario dell'Esercizio

Vi vengono forniti 3 casi di spec-driven development dove l'implementazione di Claude non soddisfa le aspettative:

Caso A: Specifica ambigua che ha portato a implementazione parzialmente corretta

Caso B: Specifica chiara ma implementazione che diverge su aspetti chiave

Caso C: Specifica e implementazione allineate ma risultato non soddisfa bisogno reale (problema nei requisiti originali)

Task per Ciascun Caso

19. Leggere specifica e implementazione
20. Diagnosticare: Il problema è nella specifica, nell'implementazione, o nei requisiti originali?
21. Fornire il feedback appropriato (riscrivere sezioni di specifica O feedback su implementazione O challenge dei requisiti)

Caso A - Specifica Problematica

Specifica fornita:

"Crea una funzione Python per validare email addresses. Deve controllare che la email sia valida e ritornare True/False."

Implementazione di Claude:

[Include funzione base che controlla presenza di @ e . ma non valida formato RFC compliant]

Domanda: Cosa è problematico nella specifica? Come la riscrivereste?

Caso B - Implementazione Divergente

Specifica fornita:

[Include specifica dettagliata per API endpoint con specifici status codes richiesti]

Implementazione di Claude:

[Claude usa status codes diversi da quelli specificati]

Domanda: Qual è il feedback da dare a Claude? Come lo formulate?

Criteri di Successo

- Diagnosi corretta della root cause per ciascun caso
- Feedback formulato in modo specifico e actionable
- Per Caso A: Specifica riscritta elimina ambiguità
- Per Caso B: Feedback cita sezioni specifiche della specifica violate
- Per Caso C: Articolazione chiara di perché requisiti originali sono problematici

3.5 Esercizio 5: Real-World Challenge (Livello Avanzato)

Obiettivi

- Applicare tutte le competenze apprese a caso realistico complesso
- Sviluppare autonomia nel troubleshooting
- Praticare decision-making sotto incertezza

Tempo

45 minuti (35 min lavoro individuale + 10 min presentazioni)

Challenge

Utilizzate la vostra istanza di Claude Desktop per completare un task complesso che voi stessi proponete (o scegliete da lista fornita). Il task deve essere:

- Rilevante per il vostro lavoro reale
- Sufficientemente complesso da richiedere 20-30 minuti con Claude
- Probabilmente presenterà almeno un problema durante l'esecuzione

Task Sugeriti (se non avete preferenze)

- Creare specifica tecnica completa per feature che state sviluppando
- Analizzare dataset complesso e produrre insights actionable
- Generare documentazione tecnica per sistema esistente
- Creare strategy document per iniziativa di business



Nota Importante

L'obiettivo NON è completare perfettamente il task ma dimostrare capacità di troubleshooting quando emergono problemi. Se tutto va liscio al primo tentativo, il task probabilmente non era sufficientemente complesso.

Criteri di Valutazione

- Approccio sistematico al troubleshooting (non trial-and-error casuale)
- Diagnosi accurata delle root causes
- Soluzioni appropriate ai problemi identificati
- Capacità di articolare learning process

4. CASI D'USO REALI

Questa sezione presenta casi di studio dettagliati da contesti professionali reali.

4.1 Case Study: Document Generation con Formatting Inconsistente

Background e Contesto

Una compliance officer deve generare mensilmente 15-20 report regolatori, ciascuno con formato specifico richiesto dalle autorità. Ha iniziato a usare Claude per accelerare il processo ma riscontra che i documenti generati hanno formatting inconsistente anche quando usa template apparentemente identici.

Situazione iniziale:

- Alcuni report hanno header corretti, altri no
- Numerazione sezioni a volte corretta, a volte parte da numero sbagliato
- Tabelle formattate diversamente tra documenti
- Necessità di cleanup manuale significativo (30-40 minuti per documento)

Journey Diagnostico

Passo 1: Caratterizzazione del problema

Analizzando multipli documenti problematici, identifichiamo pattern:

- Inconsistenza non è casuale - correlata a lunghezza del content source
- Documenti più lunghi hanno maggiori problemi di formatting
- Claude sembra 'dimenticare' istruzioni di formatting verso fine di documenti lunghi

Passo 2: Formulazione ipotesi

Ipotesi primaria: Context window che si satura durante generazione di documenti lunghi causa perdita di 'awareness' delle istruzioni di formatting.

Passo 3: Testing

Test condotto: Generare documento corto (3 pagine) vs. lungo (15 pagine) con identico prompt. Risultato: Documento corto ha formatting perfettamente consistente, documento lungo mostra degradazione dopo pagina 8-9.

Conferma: Problema è context saturation.

Soluzione Implementata

Approccio multi-step con template rigido:

Step 1: Creare template di formatting come documento separato salvato nel Progetto

Step 2: Per documenti lunghi, generare per sezioni:

- Conversazione A: Generare sezioni 1-3
- Conversazione B: Generare sezioni 4-6
- Conversazione C: Generare sezioni 7-9

- Ciascuna conversazione carica il template di formatting

Step 3: Assembly finale con check di consistency

Risultati

- Formatting consistency: 98% (vs. 60% precedente)
- Tempo cleanup manuale: 5 minuti per documento (vs. 35 minuti)
- Tempo totale per documento: Ridotto del 65%
- Bonus: Approccio modulare facilita updates quando requisiti cambiano

Lezioni Apprese

22. Consistency in output lungo-forma richiede gestione esplicita di context
23. Template separati e referenziati > istruzioni inline lunghe
24. Decomposizione preemptive > troubleshooting reattivo
25. Per task ripetitivi, investimento in setup di process robusto ha ROI alto

4.2 Case Study: Code Generation con Repeated Errors

Background e Contesto

Un team di developers usa Claude per generare boilerplate code per microservices. Notano che Claude ripetutamente fa gli stessi errori specifici anche dopo correzione esplicita.

Errori ricorrenti:

- Mancata gestione di edge case specifici (null values, empty arrays)
- Uso di librerie deprecated nonostante indicazioni contrarie
- Pattern di error handling inconsistenti con codebase esistente

Journey Diagnostico

Problema diagnosticato: Prompt non fornisce esempi concreti. Claude 'capisce' istruzioni verbali ma non le applica consistentemente senza pattern da seguire.

Soluzione

Creazione di 'Reference Implementation' come parte del progetto:

- 2-3 microservices esistenti ben fatti caricati come reference
- Prompt esplicita: 'Segui pattern e convenzioni in reference implementations'
- Checklist di code review automatizzata

Risultati

- Errori ricorrenti eliminati: 95% reduction
- Code review time: Ridotto del 40%
- Consistency cross-microservices: Migliorata significativamente

Lezione chiave: Examples > Instructions per consistency.

4.4 Case Study: Multi-Step Workflow Failures

Background

Un product manager vuole automatizzare workflow: ricerca competitiva → analisi SWOT → raccomandazioni strategiche. Workflow fallisce costantemente a step 2-3.

Diagnosi

Output di step 1 è troppo voluminoso e non strutturato, rendendo impossibile per step 2 di processarlo efficacemente.

Soluzione

Redesign workflow con output strutturati espliciti:

- Step 1: Output in formato JSON con schema definito
- Step 2: Input validation prima di procedere
- Step 3: Template output con placeholder da riempire

Lezione: Multi-step workflow richiedono output strutturati e interfacce chiare tra step.

5. BEST PRACTICES E LINEE GUIDA

5.1 Framework Diagnostico Completo

Checklist sistematica per troubleshooting qualsiasi problema:

FASE 1: OBSERVATION

- ☐ Cosa esattamente non funziona? (Sii specifico)
- ☐ È consistente o intermittente?
- ☐ Quando è iniziato?
- ☐ È peggiorato nel tempo?

FASE 2: CONTEXT

- ☐ Verifica prompt esatto
- ☐ Verifica file/context caricati
- ☐ Verifica lunghezza conversazione
- ☐ Verifica configurazione progetto

FASE 3: HYPOTHESIS

- ☐ Quale categoria? (Comprensione/Qualità/Inconsistenza/Appropriatezza/Performance)
- ☐ Quali sono 2-3 ipotesi più probabili?
- ☐ Quale è testabile più facilmente?

FASE 4: TEST

- ☐ Modifica UN elemento
- ☐ Testa in condizione controllata
- ☐ Confronta risultato con baseline

FASE 5: SOLUTION

- ☐ Applica fix mirato
- ☐ Verifica risoluzione
- ☐ Documenta per future reference

5.2 Pratiche Preventive

Prevenire problemi è 10x più efficiente che risolverli:

PROMPT ENGINEERING PREVENTIVO

- Inizia sempre con prompt strutturato e completo
- Include esempi quando possibile
- Specifica formato output desiderato
- Definisci criteri di successo espliciti

CONTEXT MANAGEMENT PROATTIVO

- Usa progetti per lavoro ongoing
- Crea nuova conversazione ogni 20-25 messaggi
- Mantieni documenti di reference nel progetto
- Fai summary periodici

SPEC-DRIVEN APPROACH

- Per task complessi, scrivi specifica prima
- Include criteri di accettazione chiari
- Review specifica prima di implementation

ITERAZIONE DISCIPLINATA

- Feedback specifico e strutturato
- Non più di 3 iterazioni senza rethinking approach
- Riconosci progress incrementale

5.3 Decision Trees per Troubleshooting Rapido

Decision tree per diagnosis rapida:

OUTPUT NON È QUELLO CHE VOLEVI?

↓

Claude ha capito la richiesta?

NO → Problema di COMPrensione → Rendi prompt più specifico

SÌ → Continua ↓

Output è consistente tra tentativi?

NO → Problema di INCONSISTENZA → Check context window

SÌ → Continua ↓

Output ha quality issues?

SÌ → Problema di QUALITÀ → Aggiungi esempi/specifiche

NO → Problema di APPROPRIATEZZA → Riconsidera se Claude è tool giusto

7. NOTE PER IL FORMATORE

7.4 Domande Comuni e Risposte Preparete

Q: "Perché dovrei imparare troubleshooting quando posso semplicemente ricominciare?"

A: Ricominciare funziona per task semplici ma diventa insostenibile per progetti complessi dove hai investito significativo tempo. Troubleshooting efficace ti fa risparmiare ore e migliora progressivamente la qualità del tuo lavoro con Claude.

Q: "Claude è non-deterministico, quindi troubleshooting non è futile?"

A: Vero che Claude ha variabilità, ma la maggior parte dei problemi sono deterministici (prompt ambigui, context issues, etc.). Troubleshooting sistematico identifica e risolve questi problemi ricorrenti.

Q: "Quanto tempo devo dedicare a troubleshooting prima di provare approccio diverso?"

A: Regola pratica: Se dopo 3 iterazioni ben ragionate non vedi miglioramento, è tempo di step back e riconsiderare approccio. Troubleshooting efficace è veloce, non infinito.

7.5 Follow-Up e Homework

Assignment consigliato post-sessione:

HOMEWORK ASSIGNMENT:

"Nelle prossime 2 settimane, tenete un 'Troubleshooting Journal':

- 26. Documentate ogni problema che incontrate con Claude
- 27. Per ciascuno, annotate: sintomo, diagnosi, soluzione, outcome
- 28. Identificate pattern: quali problemi sono ricorrenti?
- 29. Alla fine delle 2 settimane, create un documento personale 'My Common Issues & Solutions'

Questo esercizio consolida learning e crea risorsa personalizzata.

8. RISORSE SUPPLEMENTARI

8.1 Quick Reference Card

Troubleshooting Quick Checklist (Stampabile)

- ☐ Prompt specifico e completo? ☐ Contesto sufficiente fornito? ☐ Formato output specificato? ☐ Conversazione troppo lunga? (>25 msg) ☐ Task appropriato per Claude? ☐ Esempi forniti quando utile? ☐ Requisiti qualità espliciti? ☐ Testato in nuova conversazione? ☐ Iterazione produttiva o loop? ☐ Root cause identificata?

8.5 Glossario Troubleshooting

Termini chiave e definizioni:

Context Window: Lo spazio di 'memoria di lavoro' di Claude, limitato, che contiene conversazione corrente, file caricati, e contesto del progetto.

Context Saturation: Condizione in cui context window è piena/quasi piena, causando degrado di performance.

Root Cause: La causa fondamentale di un problema, distinta dai sintomi superficiali.

Iteration Loop: Pattern problematico dove iterazioni multiple non producono miglioramenti incrementali.

Prompt Ambiguity: Elementi di vaghezza o ambiguità in un prompt che causano interpretazioni multiple.

Task Appropriateness: Valutazione se un task è adatto per essere completato da Claude vs. richiedere altri strumenti.

Spec-Driven Development: Metodologia dove si scrive specifica dettagliata prima di chiedere implementazione.